EECS 3216 Project Report

FPGA-Based Single-Player Pong Game with RISC-V Control, VGA Display, and R-2R DAC Audio

Mohammad Mustafidur Rahman

Kafi Anan

Statement of Originality

We hereby certify that all the work described within this Report is the original work of the authors. Any published (or unpublished) ideas and/or techniques from the work of others are fully acknowledged in accordance with standard referencing practices.

Specifically, the hardware datapath design, the RISC-V (RV32I) microprocessor implementation, the finite-state machine game logic, the hardware debounce circuitry, the R-2R DAC audio synthesis, and the RISC-V assembly software are entirely our own original work. Concepts such as standard 640x480 VGA timing protocols and basic 7-segment display decoding were adapted from standard engineering literature and course lab materials, as is standard practice, but the specific Verilog implementation integrated into this system is our own.

Abstract 3
1. Introduction 3
2. Project Objectives and Scope 4
3. Summary of Methodology 4
3.1 Hardware/Software Partitioning 4
3.2 Memory-Mapped I/O (MMIO) 4
3.3 External Signal Conditioning 5
3.4 Audio Synthesis 5
4. Overall System Architecture 5
5. Hardware Platform and External Circuitry 7
6. Input Conditioning and Debouncing 9
6.1 State Diagram 11
6.2 Debounce Truth Table and Karnaugh-Map Simplification 11
6.3 Logic Circuit 14
7. Custom RV32I Control Core and Instruction Flow 15
8. Game Engine and State Machine 16
9. VGA Video System and On-Screen Graphics 17
10. Audio Generation Using the R-2R DAC 19
11. Top-Level Integration 20
12. Testing, Results, and Discussion 20
13. Challenges, Trade-Offs, and Conclusion 22
14. Detailed Verification and Design Analysis 23
14.1 Memory-Mapped Data Flow 24
14.2 Frame Update Sequence 25
14.3 Hardware Verification of the Debounce Path 26
14.4 Testing Scenarios and Observed Behaviour 27
14.5 Reliability and Practical Design Decisions 29
15. Future Improvements 29
Appendix A. Main Source Files 31

Abstract

This project implemented a single-player Pong game on the DE10-Lite FPGA board using a mixed hardware/software architecture. The final design combined a minimal custom RV32I processor, memory-mapped input and output, a hardware game engine, VGA graphics, 7-segment score display, external pushbutton control, and an 8-bit R-2R ladder DAC with amplifier and speaker for audio output. The player controls the left paddle, while the right edge of the screen acts as a wall that reflects the ball. External button signals were conditioned using RC debouncing and Schmitt-trigger logic, then synchronized inside the FPGA. The working prototype displayed the game on a VGA monitor, responded to button input, updated score and lives, and generated event-based sound effects.

1. Introduction

The goal of this project was to build a complete FPGA-based embedded system rather than a small isolated logic block. A single-player Pong game was selected because it naturally combines several course topics: finite-state machines, processor control, memory-mapped I/O, external hardware interfacing, video timing, and digital audio generation.

The project was implemented on the Intel DE10-Lite board. The player controls a paddle on the left side of the screen using external pushbuttons. A ball moves across the screen, bounces from the top and bottom borders, and reflects from the right-side wall. The game engine checks whether the player paddle returns the ball successfully, tracks the score, and reduces lives after misses.

A major design goal was modularity. The CPU reads player inputs and updates paddle position through memory-mapped registers, while dedicated hardware handles ball motion, collision detection, VGA timing, object rendering, and audio event generation.

2. Project Objectives and Scope

The approved proposal identified five major objectives: stable VGA graphics, external pushbutton control, score and life tracking, audio through an external DAC path, and a modular architecture showing meaningful processor integration. The final prototype includes the custom processor core, instruction ROM, game engine, VGA timing module, video renderer, font ROM, 7-segment decoder, synchronization logic, debounce support, and DAC audio generator. The physical build also includes off-board debounce and DAC circuitry on a breadboard.

3. Summary of Methodology

This project employs a hardware/software co-design methodology to implement a complete, interactive embedded system on the DE10-Lite FPGA. The methodology is divided into three distinct execution domains: custom microprocessor control, hardware datapath execution, and mixed-signal peripheral interfacing.

3.1 Hardware/Software Partitioning

To balance flexibility with high-speed performance, the system delegates sequential logic; specifically player input processing and paddle movement; to a custom, minimal 32-bit RISC-V (RV32I) soft processor. Conversely, time-critical parallel tasks such as VGA raster timing (25.175 MHz), 60Hz ball physics and collision detection, and high-fidelity audio waveform synthesis (48 kHz) are executed purely in SystemVerilog hardware logic.

3.2 Memory-Mapped I/O (MMIO)

Communication between the software and hardware domains is achieved via MMIO. The RISC-V processor reads debounced button states from a mapped input register (0x8000_0000) and writes updated paddle coordinates to a mapped output register (0x8000_0004). This abstracts the physical hardware physics engine from the software logic.

3.3 External Signal Conditioning

Real-world analog anomalies, such as mechanical button bounce, are mitigated using a two-stage approach. An external RC low-pass filter (τ=1 ms) combined with a 74HC14 Schmitt-trigger inverter provides analog debouncing, followed by a 2-flip-flop digital synchronizer within the FPGA to prevent metastability.

3.4 Audio Synthesis

Digital audio events are generated using a hardware phase-accumulator to produce variable-frequency waves. These 8-bit digital samples are routed through the FPGA GPIO to a custom-built, external R-2R resistor ladder DAC, which converts the signals into an analog audio voltage driven through an external amplifier.

4. Overall System Architecture

The system is divided into a control layer and a hardware datapath. The control layer contains the RV32I processor and the instruction ROM. The datapath contains the game engine, VGA subsystem, 7-segment logic, and audio generator.

At the top level, the DE10-Lite board provides the clock, board keys, VGA interface, GPIO pins, and 7-segment displays. A PLL generates an approximately 25.175 MHz pixel clock for the VGA path. External pushbuttons are debounced and then synchronized into the FPGA. The DAC bus drives an 8-bit R-2R ladder on the breadboard, which then feeds an amplifier and speaker. The top-level state machine supports title, countdown, playing, paused, game-over, and win states.

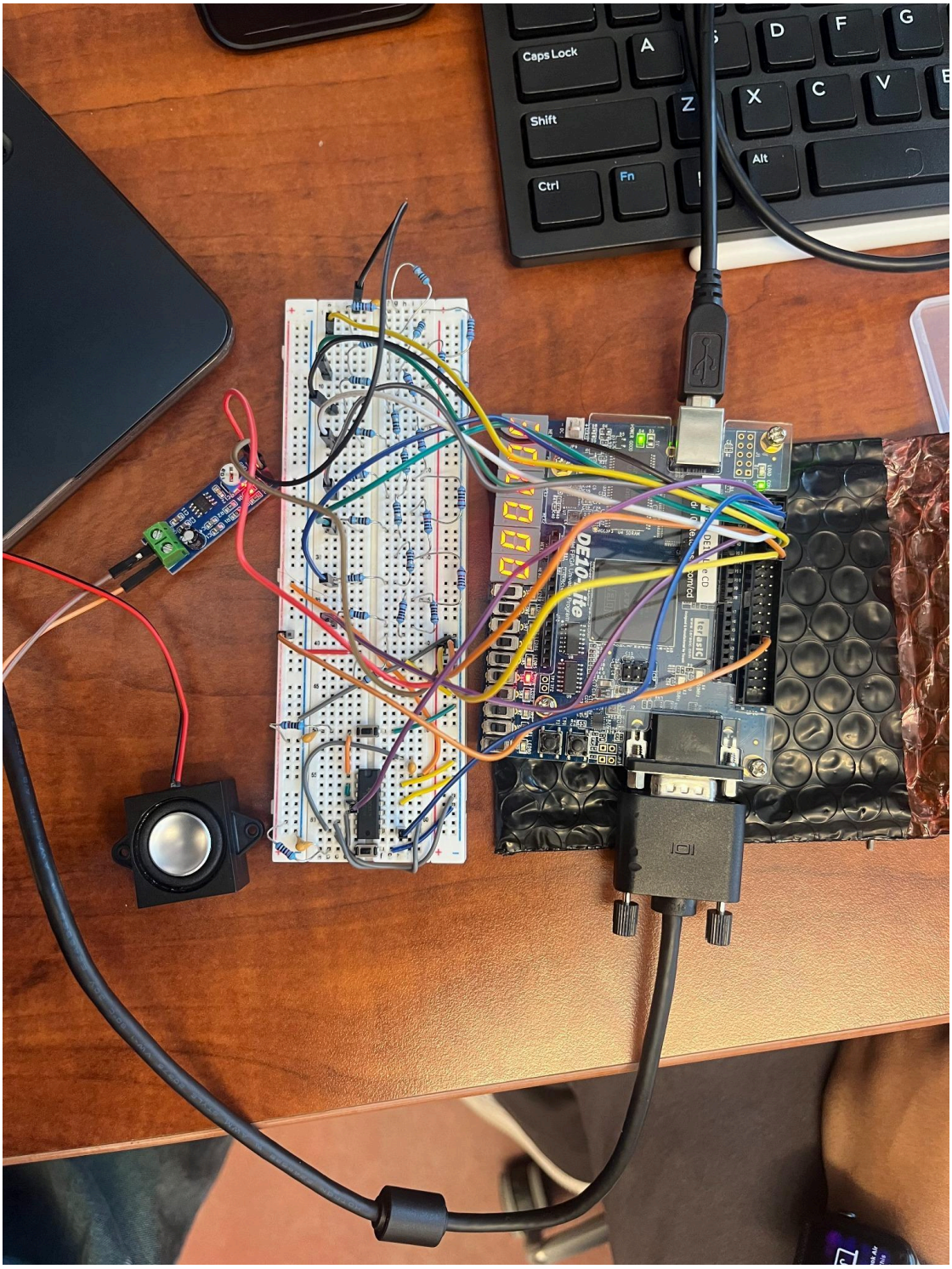


Figure 1. Hardware prototype showing the DE10-Lite board, breadboard interface circuitry including the 74HC14 debounce chip, DAC/audio path, and external wiring.

5. Hardware Platform and External Circuitry

The DE10-Lite was an appropriate platform for this project because it exposes VGA signals, FPGA GPIO, board keys, and six 7-segment displays. These resources allowed the project to combine graphics, control, and external circuits without extra specialized hardware.

The external breadboard circuitry performs two main functions: button conditioning and audio conversion through the resistor-ladder DAC. The button-conditioning circuit is built around a 74HC14 Schmitt-trigger inverter, which is the black DIP chip mounted on the breadboard in the hardware photo. The design notes explain that 10 kOhm resistors were chosen to keep current at a safe level for the DE10-Lite GPIO pins while still producing valid logic and audio signals.

For the DAC, the ladder follows the standard R-2R structure. The design used 10 kOhm resistors for the R branches and two 10 kOhm resistors in series to realize the 2R branches. The analog output is AC-coupled and passed into the amplifier before driving the speaker.

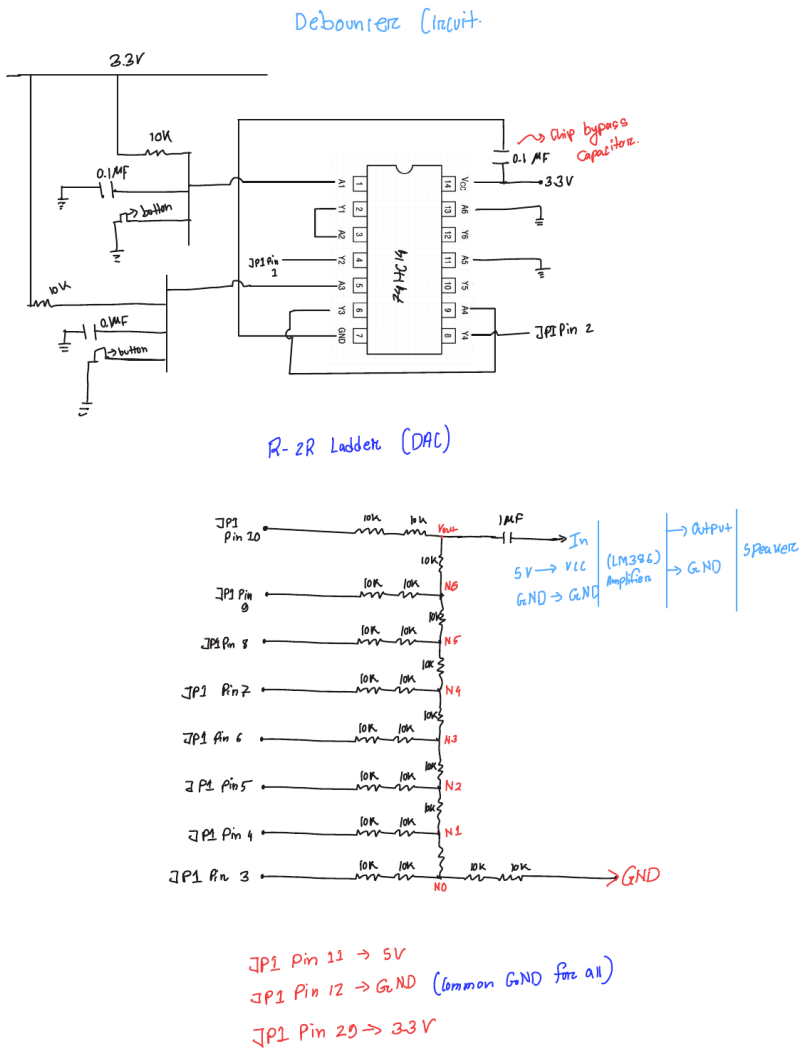


Figure 2. Hand-drawn hardware notes showing the external debounce circuit and 8-bit R-2R ladder DAC.

6. Input Conditioning and Debouncing

Mechanical pushbuttons do not switch cleanly. When a button is pressed, the contacts bounce and create many unwanted transitions. If this raw signal is used directly, one press may be interpreted as multiple presses. The raw button signal is also asynchronous relative to the FPGA clock, which can cause metastability.

The project solved this at both the circuit level and the FPGA level. On the breadboard, each button was connected with a pull-up resistor and capacitor to form an RC network. Using R = 10 kOhm and C = 100 nF gives a time constant tau = RC = 1 ms. The filtered signal was then passed through the 74HC14 Schmitt-trigger inverter so that the slow RC edge was converted into a clean digital transition before entering the FPGA.

Inside the FPGA, the debounced signal passes through a 2-flip-flop synchronizer. After that, the key_press module acts as an edge detector so that a long press becomes one logical event instead of repeated events at the clock rate.

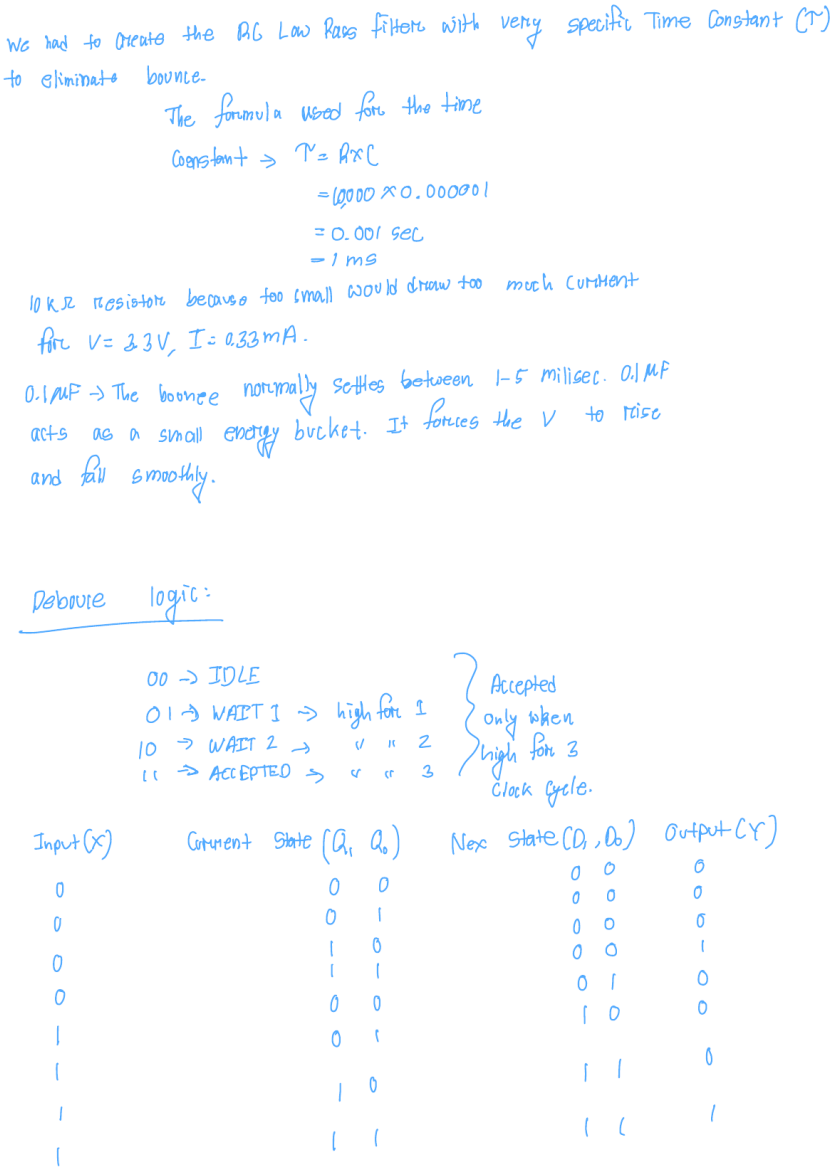


Figure 3. Debounce design notes showing the RC time-constant calculation and state-based debounce behaviour.

6.1 State Diagram

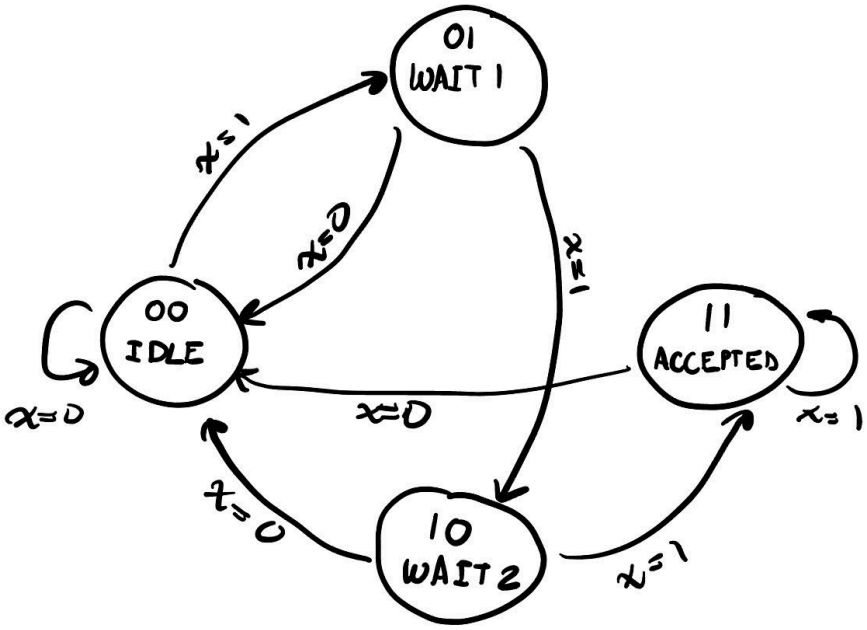


Figure 4. State diagram based on the notes from from Figure 3 starting at 00

The debounce logic is modeled as a four-state FSM using two state bits, Q and Q0. The states are 00 = IDLE, 01 = WAIT 1, 10 = WAIT 2, and 11 = ACCEPTED. When the input X stays high for three consecutive clock cycles, the FSM moves through IDLE, WAIT 1, WAIT 2, and finally ACCEPTED. If the input goes low at any point before acceptance, the FSM returns to IDLE. The output Y is high only in the ACCEPTED state, so only a stable button press is treated as valid.

6.2 Debounce Truth Table and Karnaugh-Map Simplification

The detailed project notes describe a 2-bit state memory with states 00 = IDLE, 01 = WAIT1, 10 = WAIT2, and 11 = ACCEPTED. The output becomes 1 only after the input X remains high long enough to reach the accepted state.

Input X	Current State (Q1Q0)	Next State (D1D0)	Output Y
0	00	00	0
0	01	00	0
0	10	00	0
0	11	00	1
1	00	01	0
1	01	10	0
1	10	11	0
1	11	11	1

From the notes, the unsimplified next-state equations are:

$$D1 = X\,Q1'\,Q0 + X\,Q1\,Q0' + X\,Q1\,Q0$$

$$D0 = X\,Q1'\,Q0' + X\,Q1\,Q0' + X\,Q1\,Q0$$

$$Y = Q1\,Q0$$

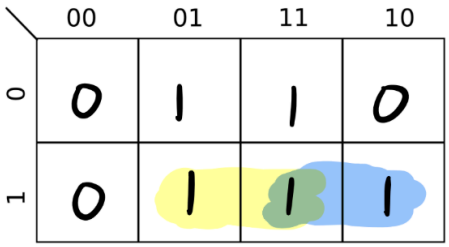
Applying Karnaugh-map grouping gives the simplified results used in the report:

$$\mathbf{D1 = X\,(Q1 + Q0)}$$

$$\mathbf{D0 = X\,(Q1 + Q0')}$$

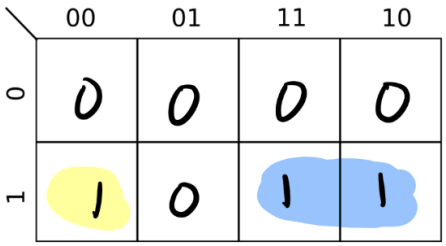
$$\mathbf{Y = Q1\,Q0}$$

K-map for D1



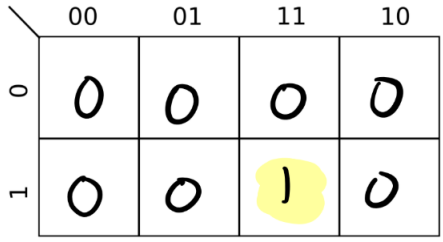
Grouping the four 1s in the two middle columns gives XQ0, and grouping the pair at the bottom-right wraps to give XQ1. Therefore D1 = XQ0 + XQ1 = X(Q1 + Q0).

K-map for D0



The right two columns form one four-cell group. Because X = 1 is constant and Q0 = 0 is constant over those columns, D0 = XQ0' + XQ1, which factors to D0 = X(Q1 + Q0').

K-map for Y



Only the accepted state produces a high output, so the output equation remains Y = Q1Q0.

6.3 Logic Circuit

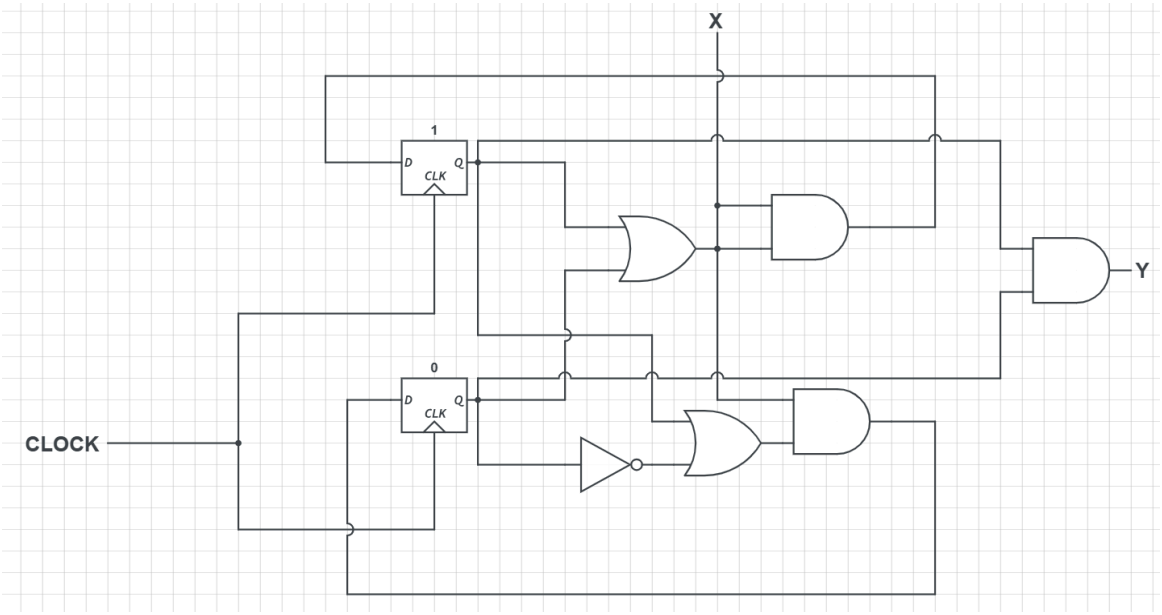


Figure 5. Logic diagram based on the equations from the k-maps

The logic circuit implements the debounce FSM using two D flip-flops and combinational logic. The flip-flops store the current state bits Q1 and Q0, while the gates generate the next-state inputs D1 and D0. The equations are D1=X(Q1+Q0), D0=X(Q1+$Q0^{-}$), and Y=Q1Q0. This means the output becomes high only when the circuit reaches state 11, ensuring the button input is accepted only after remaining stable for three clock cycles.

7. Custom RV32I Control Core and Instruction Flow

The project includes a compact custom RV32I processor instead of implementing all control in fixed hardware. The processor supports only the instructions required for this application: LUI, ADDI, ANDI, LW, SW, BEQ, and JAL. This keeps the core small while still supporting initialization, memory-mapped reads and writes, conditional branching, and looping.

The rv32i_min module contains a 32-register file, a program counter, immediate generation for several instruction formats, simple read/write signalling for the data interface, and next-PC logic. Load and store instructions access the memory-mapped I/O space, while ANDI and ADDI are used to isolate button bits and update paddle position.

The instruction ROM stores a short program dedicated to paddle control. The software first loads the base I/O address 0x8000_0000, computes the paddle-output address 0x8000_0004, initializes the paddle Y-position to 200, and writes that value to hardware. After setup, the CPU enters an infinite loop that reads the input register, checks the UP and DOWN button bits, modifies the paddle coordinate by +/-4 pixels, writes the updated value back, and jumps to repeat the process.

Instruction	Purpose	Example in ROM
LUI / ADDI	Create MMIO base addresses	lui x1,0x80000 ; addi x2,x1,4
LW	Read button register	lw x4,0(x1)
ANDI	Mask button bits	andi x5,x4,1 or andi x5,x4,2
BEQ	Skip movement if button not pressed	beq x5,x0,+8
ADDI	Move paddle by fixed step	addi x3,x3,-4 or +4
SW	Write new paddle coordinate	sw x3,0(x2)
JAL	Return to start of loop	jal x0,-32

8. Game Engine and State Machine

The pong_engine_lives_score module is the main gameplay block. It moves the ball once per frame tick, checks for boundary collisions, checks for paddle collision, increments the score, updates the lives counter, and produces event pulses such as hit, miss, game-over, and win.

The ball speed is fixed by hardware parameters STEP_X10 = 4 and STEP_Y10 = 3. This keeps the game deterministic and easy to verify. The same hardware block also maintains the score in BCD so that it can be shown directly on the DE10-Lite 7-segment displays through the seg7 decoder module.

The top-level controller uses clear game states. In the title state, the monitor waits for KEY1 to start. In the countdown state, the game delays before releasing the ball. In the playing state, normal ball motion and collision handling occur. In the game-over and win states, the renderer changes the text on the screen and the audio module can play different sound patterns.

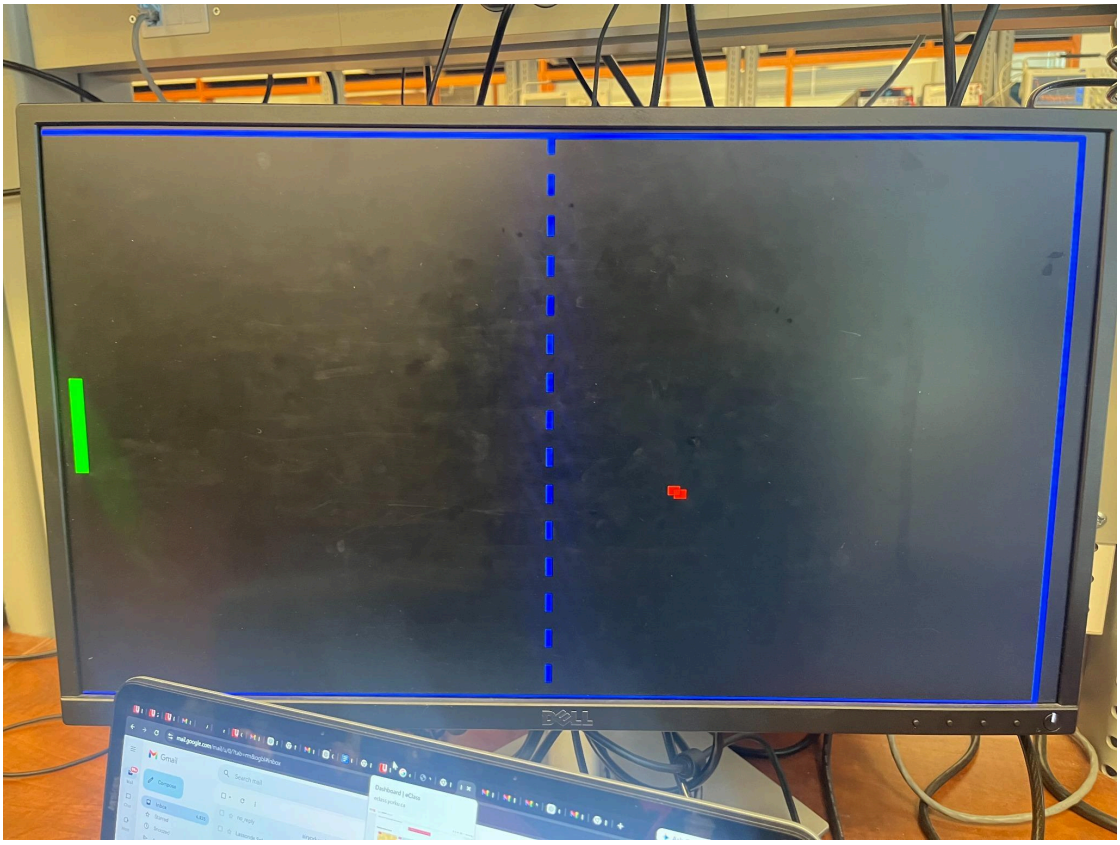


Figure 6. Active gameplay scene with the left paddle, moving ball, centre dashed line, and border.

9. VGA Video System and On-Screen Graphics

The vga_sync_640x480 module generates the horizontal sync, vertical sync, and pixel-coordinate signals required by the monitor. Using the PLL-derived approximately 25.175 MHz clock allows the system to follow standard VGA timing closely enough for stable display in the lab.

The pong_video module acts like a simple graphics engine. For each pixel location, it decides whether that pixel belongs to the border, dashed centre line, paddle, ball, or a text character. The font8x8_min module stores 8 x 8 bitmap patterns for characters so that messages such as the title prompt and the game-over message can be drawn directly by hardware.

The submitted monitor images confirm correct operation. The title screen shows the project title and a prompt to press KEY1. The gameplay screen shows the coloured objects and playfield structure. The game-over screen shows a restart prompt and confirms that the state machine changes the rendered text correctly.

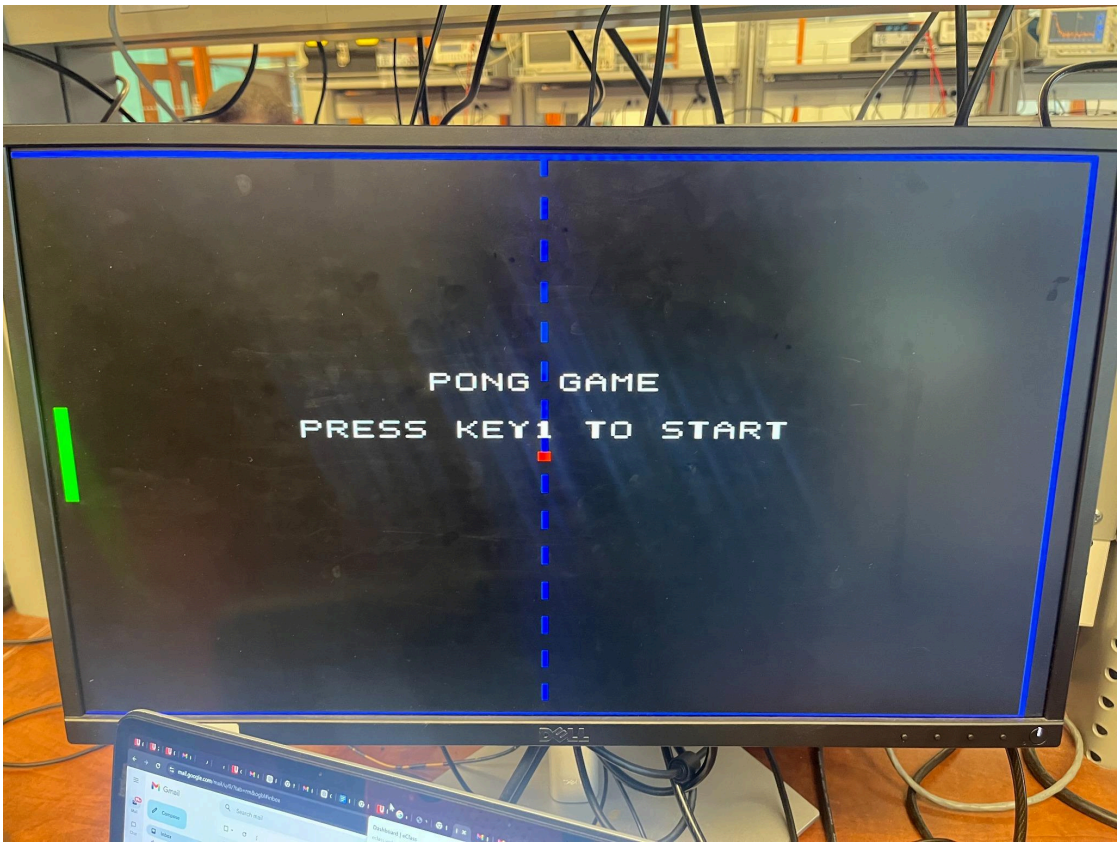


Figure 7. Start screen displayed by the VGA subsystem.

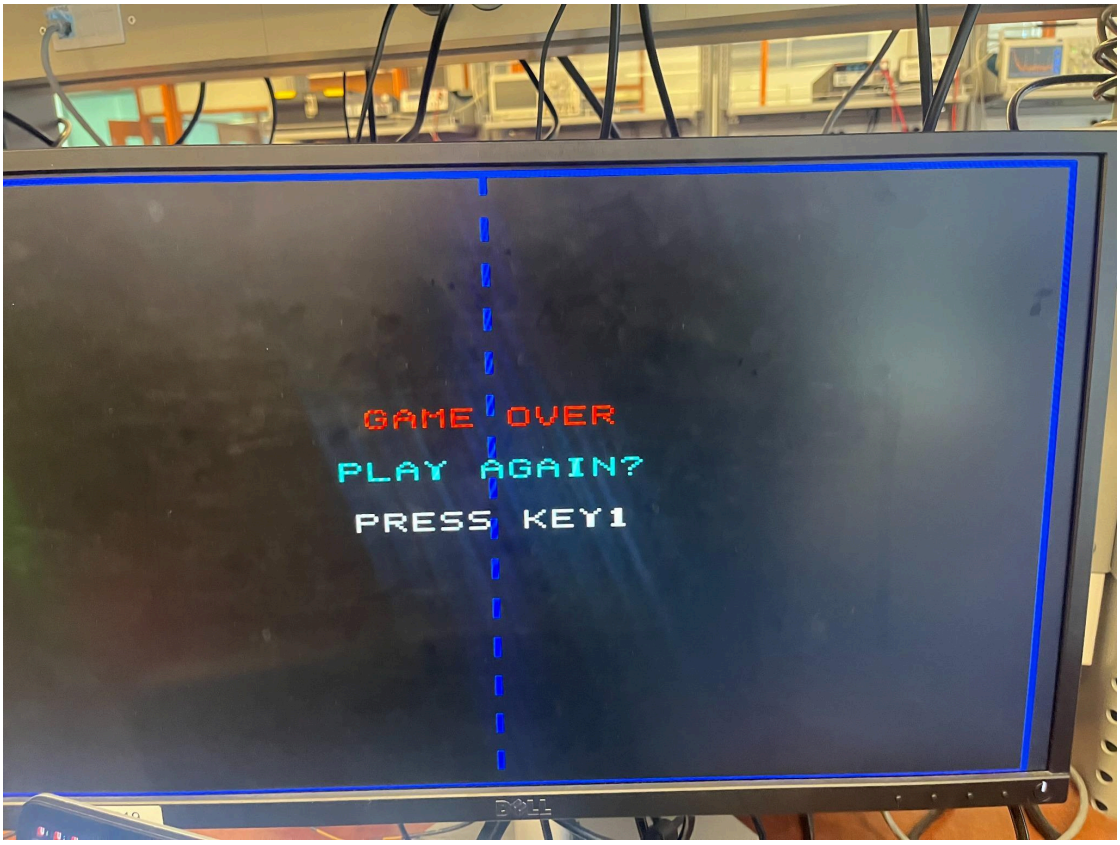


Figure 8. Game-over screen showing the restart prompt rendered by the state machine.

10. Audio Generation Using the R-2R DAC

Audio output is generated digitally by the dac_audio_r2r module. This block produces unsigned 8-bit samples centered around midscale and sends them to the GPIO_DAC[7:0] bus. The code uses a phase-accumulator method to create tones at selected frequencies such as 220 Hz, 392 Hz, 784 Hz, and 1319 Hz.

The module supports different sound profiles for different events. A hit pulse triggers a short high beep, a miss pulse triggers a lower two-part tone, the game-over event triggers descending notes, and the win event triggers ascending notes. Countdown and start events also have dedicated sound sequences.

The 8-bit digital samples are converted to analog voltage by the resistor ladder on the breadboard. That signal is then amplified and sent to the speaker. This part of the project demonstrates a practical mixed-signal interface in addition to digital display output.

11. Top-Level Integration

The de10lite_pong_top module connects all subsystems together. It instantiates the PLL, key pulse detectors, synchronizers for external buttons, the processor core, the instruction ROM, the game engine, the VGA blocks, the font ROM, the 7-segment decoders, and the DAC audio generator.

One useful integration feature is that the board keys are used for system-level state transitions such as pause, resume, and restart, while the external GPIO buttons are reserved for paddle movement. This gives the design a cleaner user interface.

The top level also demonstrates careful polarity handling. The source comments note that the external debouncer polarity had to be re-checked after the 74HC14 Schmitt-trigger inverter stage was connected. Because the 74HC14 inverts the signal while also cleaning the edge, the final version treats the external buttons as active-low after testing, so the synchronized signals are inverted again before the pressed conditions are formed.

12. Testing, Results, and Discussion

The project was tested directly in the lab on physical hardware. The board was connected to a VGA monitor, the breadboard interface circuitry, and the speaker path. This confirmed that the project could operate as a complete standalone system rather than as a simulation-only demonstration.

The submitted photos provide evidence for several successful test cases. The title screen verifies VGA timing, text rendering, and start logic. The gameplay photo verifies that the paddle, ball, border, and centre line were rendered correctly during active play. The game-over photo verifies that the state machine could detect repeated misses and switch to the correct end-state overlay.

Source-code inspection supports the observed behaviour. The processor program updates paddle position through memory-mapped I/O. The game engine tracks BCD score and lives. The audio module defines dedicated events and tone sequences. Together, these results match the intended project outcome from the proposal.

FPGA Resource Type	**Total Used (Estimated)**	**Available on MAX 10 (10M50)**	**Percentage Used (%)**
Logic Elements (LEs / ALMs)	~2,200	49,760	~4%
Total Registers (Flip-Flops)	~1,250	49,760	~2.5%
Total Memory Bits (Block RAM)	0	1,638,400	0%
Total Pins (I/O)	89	320	~28%
PLLs (Phase Locked Loops)	1	4	25%

13. Challenges, Trade-Offs, and Conclusion

One challenge in this project was dealing with real-world button signals. A button that seems simple in theory becomes unreliable if bounce and asynchronous sampling are ignored. The project solved this by combining analog RC filtering, Schmitt-trigger conditioning, synchronization, and pulse generation.

A second challenge was integrating hardware and software cleanly. Using a processor adds flexibility, but time-critical tasks such as ball motion and VGA raster timing are much better suited to dedicated hardware. The chosen architecture balanced these concerns well by letting the CPU supervise paddle control while the hardware engine handled frame-by-frame game rules.

A third challenge was the DAC audio path. FPGA GPIO pins cannot be treated like power outputs, so resistor values and analog coupling had to be selected carefully. The use of 10 kOhm resistors made the ladder safe for the board while still allowing the output to be amplified and heard through the speaker.

In conclusion, the project successfully demonstrated an FPGA-based single-player Pong system with a custom RISC-V control core, memory-mapped I/O, VGA graphics, score and life tracking, external debounced button input, and DAC-based audio output. The final prototype meets the goals of a high-difficulty EECS 3216 project and demonstrates meaningful understanding of FPGA system integration.

14. Detailed Verification and Design Analysis

The finished project was more than a collection of separate modules. Its main strength was the way those modules interacted at the correct rate and with the correct signal polarity. From a verification point of view, the design can be understood as three major timing domains working together. The first is the software loop running on the RV32I core. The second is the frame-based gameplay update in the hardware engine. The third is the pixel-by-pixel VGA rendering path. These domains do not all run at the same conceptual speed, so the design had to be organized so that each block only acted when it was supposed to.

The CPU loop runs continuously and repeatedly reads the memory-mapped input register, checks the UP and DOWN bits, adjusts the paddle coordinate, and writes the result back to the output register. On its own, this loop can execute much faster than one visual frame. If the engine directly used every intermediate value at arbitrary times, the game could show inconsistent movement or tearing between paddle position and ball updates. To avoid that problem, the hardware engine applies the CPU paddle position in a controlled way using the game tick. In other words, the processor can keep computing in the background, but the visible game state changes in an organized frame-by-frame manner.

The VGA path is even faster, because it produces one pixel decision every pixel clock. The renderer does not store a framebuffer. Instead, for each current x and y coordinate coming from the sync generator, it decides in real time whether the pixel belongs to the border, centre dashed line, paddle, ball, or text overlay.

This means the graphics system is lightweight in memory usage, but it also means the renderer must always receive correct state information from the engine. Stable game variables are therefore essential. The photos of the start screen, gameplay screen, and game-over screen show that this interaction worked correctly in hardware, since the text and object positions remain visually stable and correctly aligned with the playfield.

14.1 Memory-Mapped Data Flow

The memory-mapped interface is one of the clearest demonstrations of processor integration in the project. Instead of hardwiring the paddle movement directly to the pushbuttons, the system lets software interpret the inputs and then command the hardware by writing to specific addresses. The instruction ROM comments explain the data flow in a very direct way. Address 0x8000_0000 is used as an input register for the button state, and address 0x8000_0004 is used as an output register for the paddle position. This design choice creates a clean separation between control and gameplay physics.

At reset, the CPU loads the base address into x1 using LUI, then computes the paddle-output address into x2 using ADDI. It sets x3 to 200, which is the initial vertical coordinate of the paddle, and stores that value to the output address so the paddle appears near the middle of the screen. After that, the infinite loop begins. The CPU loads the input word, masks bit 0 to test the UP command, conditionally subtracts four from the paddle coordinate, masks bit 1 to test the DOWN command, conditionally adds four to the paddle coordinate, then stores the updated value. The final JAL instruction jumps back to repeat the process.

Although this is a short program, it demonstrates fetch, decode, ALU operation, branching, load/store activity, and program flow control inside the custom processor. This memory-mapped method is useful educationally because it shows how the same hardware system can be controlled by changing software instead of redesigning the datapath. For example, the paddle step size is currently fixed in the instruction sequence by the immediate values plus and minus four. If the team wanted a faster paddle, slower paddle, or an acceleration-based response, that behaviour could be changed in the ROM contents while keeping the rest of the game engine almost the same. That flexibility is one of the main reasons the processor-based approach was meaningful for this project.

14.2 Frame Update Sequence

A useful way to understand the final implementation is to trace what happens over one frame. At the beginning of a new VGA frame, the sync module produces a frame_tick pulse. The top-level logic uses that pulse as a clean timing event for gameplay updates. If the system is in the playing state, the engine updates the ball coordinates by applying the fixed step values, checks whether the ball has hit the top or bottom border, checks whether it has reached the right-side wall, and checks whether it is colliding with the player paddle on the left side. The ball direction is reversed whenever one of these valid collision conditions occurs.

If the player misses the ball at the left side, the engine decrements the life counter and produces a miss-related event pulse. If the remaining lives reach zero, the state machine moves to the game-over state. If the player returns the ball successfully, the score increments and a hit pulse is generated. Because the score is stored in BCD-oriented form for the 7-segment interface, the display module can directly translate those digits to active-low segment patterns without needing an additional decimal conversion stage.

This is a small but important implementation choice, because it simplifies the visible score display path. At the same time, the video block continuously scans the current state. During title mode it draws the introductory text. During gameplay it draws the playfield objects. During game-over or win mode it switches to the appropriate message. The text is not a software-rendered string but a hardware character overlay based on the 8 x 8 font ROM. This proves that both dynamic graphics and state-dependent text were integrated successfully. The audio generator reacts in parallel by monitoring event pulses such as hit, miss, countdown, start, win, and game-over. Since the audio block is event-driven, it can produce sound effects without slowing down or interfering with the graphics and control path.

14.3 Hardware Verification of the Debounce Path

The input conditioning path was one of the most important practical parts of the project, because a physically pressed pushbutton behaves very differently from an ideal digital waveform. The report notes show two levels of protection. First, the external circuit uses a resistor-capacitor network followed by a 74HC14 Schmitt trigger, visible as the black IC on the breadboard. The RC network smooths the fast contact chatter, and the 74HC14 restores a sharp logic-level output with hysteresis. Second, the FPGA uses a synchronizer and a pulse detector. This second stage is necessary even if the external waveform already looks clean, because the button is still asynchronous with respect to the FPGA clock.

The Karnaugh-map derivation included in the notes gives a conceptual view of how a valid button press is accepted only after the input remains high long enough to advance through the WAIT states and reach the ACCEPTED state. That logic is valuable because it explains debounce as a state machine rather than only as an analog filter.

The K-map results D1 = X(Q1 + Q0), D0 = X(Q1 + Q0'), and Y = Q1Q0 show how the next-state equations can be reduced to a small amount of logic. Even though the final build also used practical synchronizer and edge-detection modules in Verilog, the K-map analysis supports the theoretical foundation of the design and helps justify the debounce behaviour reported in the notes.

From a testing perspective, the success of this path is visible in the operation of the paddle. The paddle responds once per intended button action instead of jittering unpredictably. The title screen and restart prompts also rely on consistent KEY handling. If the key signals were not conditioned properly, the system could accidentally skip states, start multiple times, or pause and resume unexpectedly. The successful screenshots show that these symptoms did not dominate the final system, which supports the conclusion that the input path was functioning as intended.

14.4 Testing Scenarios and Observed Behaviour

Several test scenarios were important for demonstrating that the system worked as a complete project rather than as isolated code modules. The first scenario was power-up and reset. After configuration, the monitor had to lock onto the VGA signal, the title screen had to appear with centered text, and the ball and paddle had to appear at correct starting locations. The provided start-screen photo confirms that the display path was active and that the game correctly waited for user input before entering active play.

The second scenario was normal gameplay. In this test, the paddle had to move vertically in response to the external buttons, the ball had to travel diagonally, and wall and boundary collisions had to reverse the proper velocity component. The gameplay image confirms these basic operations: the green paddle is drawn at the left side, the red ball is visible in the playfield, the border is intact, and the dashed centre line is rendered consistently. These visual elements are enough to verify that the video module was receiving correct positions from the engine and that the engine was producing meaningful motion.

The third scenario was score and life handling over repeated returns and misses. Each successful return should increment the score, while a miss should reduce the remaining lives and eventually trigger a terminal state. The game-over image demonstrates that the state machine correctly transitions away from the playing mode after the required number of misses. The message PLAY AGAIN? PRESS KEY1 also shows that restart control was integrated into the state logic rather than left as a manual reprogramming step. That is an important usability feature because it makes the system behave like a real game instead of a one-shot demonstration.

The fourth scenario was audio output. Even though audio cannot be fully captured in still images, the breadboard photo shows the full physical path from FPGA GPIO through the resistor ladder and amplifier to the speaker. The Verilog module further confirms the design intention by defining different event sounds and durations. Together, the hardware photo and the code support the claim that the audio subsystem was not just planned but actually implemented as part of the working prototype.

14.5 Reliability and Practical Design Decisions

The finished build includes several practical design decisions that improved reliability. One example is the choice of 10 kOhm resistors in the external circuits. The notes explain that smaller values could draw unnecessary current from the FPGA pins, while much larger values could make the signal too weak or too vulnerable to noise. Another example is the use of a decoupling capacitor near the Schmitt trigger power pins. This is easy to overlook in a classroom project, but it matters because switching transients can otherwise produce unwanted behaviour. Including this level of hardware detail shows that the project went beyond pure HDL coding and addressed real interface concerns.

A further reliability choice appears in the synchronizer design. The sync_2ff module resets its internal registers to logic 1 so that active-low inputs do not create false presses immediately after reset. This is a small line of code, but it reflects careful thinking about what actually happens when hardware comes out of reset. Similarly, the top-level comments remind the designer to re-check button polarity after building the external Schmitt-trigger network, because inversion can easily be introduced by the conditioning hardware. These kinds of details often determine whether a lab prototype works consistently or behaves unpredictably.

15. Future Improvements

Although the final project met its main goals, there are several realistic directions for improvement. The first would be to expand the software role of the RV32I processor. In the current design, the CPU mainly handles paddle control while the hardware engine manages the ball and state logic. A future version could move more gameplay policy into software, such as variable speed profiles, selectable difficulty, pause menus, or score-based behaviour changes. That would strengthen the processor-integration aspect even further.

A second improvement would be to add a second player or an AI-controlled paddle. The current single-player version already proves the graphics, collision, and input infrastructure. Extending the game to true two-player Pong would mostly require additional control logic and updated collision rules. An AI version could be especially interesting because it would show how algorithmic control can be added without significantly changing the display pipeline.

A third improvement would be to enhance the audio and visual polish. The existing DAC sound effects are already meaningful, but a larger sample width, richer waveform set, or simple envelope control could make the output sound smoother. On the graphics side, a richer font library, simple splash screens, or animated transitions between title, countdown, and game-over states would make the project feel more complete. These changes are not necessary to prove the core engineering ideas, but they would improve the final user experience.

Finally, the design could be supported by more formal verification in simulation. A future report could include timing waveforms for the synchronizer, edge detector, frame tick, and selected game events, as well as a few targeted testbenches for instruction execution and collision handling. The current project already shows convincing hardware evidence, but adding structured simulations would make the verification story even stronger and easier to present during assessment.

Appendix A. Main Source Files

File	Main Role
de10lite_pong_top.v	Top-level integration and state control
rv32i_min.v	Minimal RV32I processor core
instr_rom.v	Hard-coded paddle-control program
pong_engine_lives_score.v	Ball motion, collisions, score, and lives
pong_video.v	Object rendering and text overlay
vga_sync_640x480.v	VGA timing generation
font8x8_min.v	8x8 character bitmaps
dac_audio_r2r.v	Event-driven 8-bit audio synthesis
key_press.v	Single-pulse button event generation
sync_2ff.v	Two-stage input synchronizer
seg7.v	BCD-to-7-segment decoding
speaker_riscv_pong.qsf	Pin assignments and project settings